

Introduction to Complexity Theory

The field of complexity theory deals with how fast one can solve a certain type of problem.

More generally it refers to how much resource does it take to execution.

The most common resources are time and memory.

Classes of Problems

Mainly, Problems can be classified into two.

⇒ Tractable, and

⇒ Intractable.

Problems that can be solvable by polynomial time algorithms are known as Tractable and they are considered as easy.

Problems that require super polynomial time is known as intractable and they are considered as hard problems.

Tractable Problems or
class P Problems.

Problems that are solvable in polynomial time comes under this class.

These are problems that can be solved in polynomial time $O(n^k)$ for some constant 'k', where 'n' is the size of the input given to the problem.

There exist a deterministic polynomial time algorithm to solve these kinds of problems.

Eg:- Matrix multiplication - $O(n^3)$

Advantage of Polynomial time Algm

1 $\Rightarrow$ Considered as efficient.

Polynomial time problems requires much less time to execute.

2 $\Rightarrow$ Problems that can be solved in polynomial time in one model can be solved in polynomial time in another.

For example, the class of problems solvable in polynomial time by the serial random access machine is the same as the class of problems solvable in polynomial time on abstract Turing machines.

3 $\Rightarrow$ These are closure properties

Problems are closed under addition, multiplication and composition.

Intractable Problems - NP class

This class includes problems that can't be solved in polynomial time but can be verified in polynomial time.

That is, if we are given a certificate of a solution, we could verify that the certificate is correct in polynomial time.

NP is the class of decision problems for which there exist a polynomially bounded non-deterministic algorithm.

Non deterministic algorithm consists of two phases.

⇒ Non deterministic guessing phase

⇒ Deterministic Verification phase.

A problem in P is also in NP , since if a problem is in P , then we can solve it in polynomial time without even being given a certificate.

$$\text{ie } P \subseteq NP$$

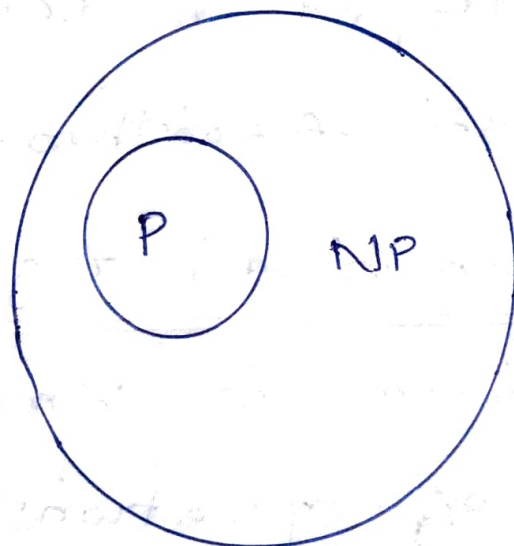


Fig:- Commonly believed relationship between P and NP .

P is the set of all decision problems solvable by deterministic algorithms in polynomial time.

NP is the set of all decision problems solvable by non-deterministic algorithms in polynomial time.

Deterministic Vs

Non Deterministic Algorithm

Deterministic Algorithm

Algorithms in which the result of every operation is uniquely defined is known as deterministic Algorithm.

Non deterministic Algorithm

Algorithms in which the outcome of operations are not uniquely defined but are limited to specific set of possibilities is known as Non deterministic Algorithm.

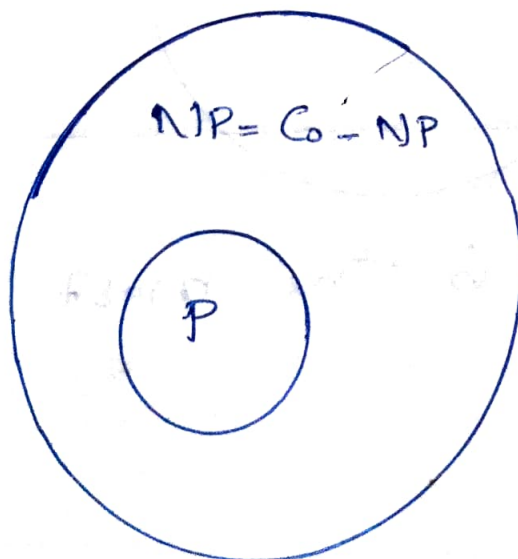
To specify such algorithms, we introduce three functions

- ① choice (s) arbitrarily chooses one of the elements of set S .
- ② Failure (f) signals an unsuccessful completion.
- ③ Success (s) signals a successful completion.

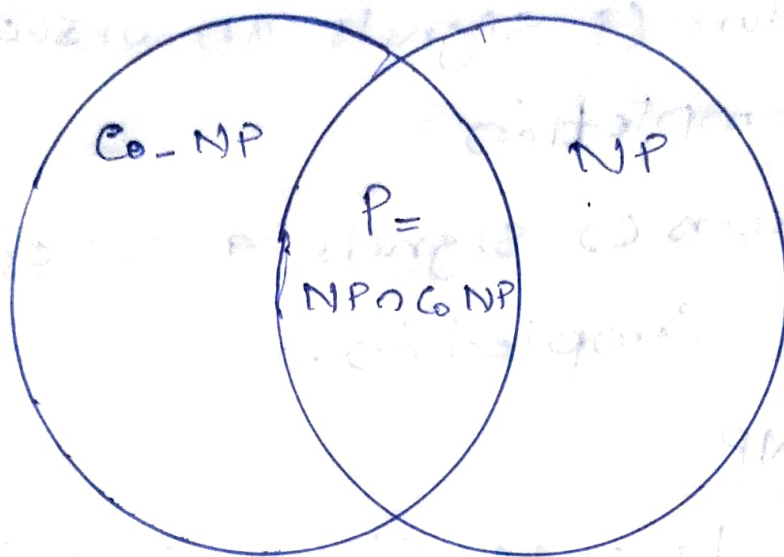
Co-NP

A language L is said to be in Co-NP if complement of the language belongs to NP.

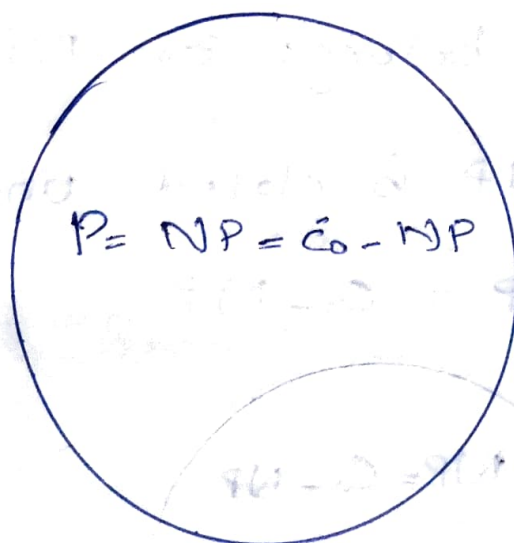
- ⑨ If NP is closed under complement then $NP = Co-NP$



⑥ If NP is not closed under complement and if $P = NP \cap \text{Co-NP}$



⑦ If $P = NP = \text{Co-NP}$



This is the most unlikely case.

NP Complete

A language is said to be in NP Complete, if it satisfies following two conditions

- ① L is in NP
- ② L is in NP hard.

i.e., it have to prove two conditions

① There exist a non-deterministic polynomial time algorithm, and

② For every problems in NP, there exist a polynomial reduction in L .

NP hard

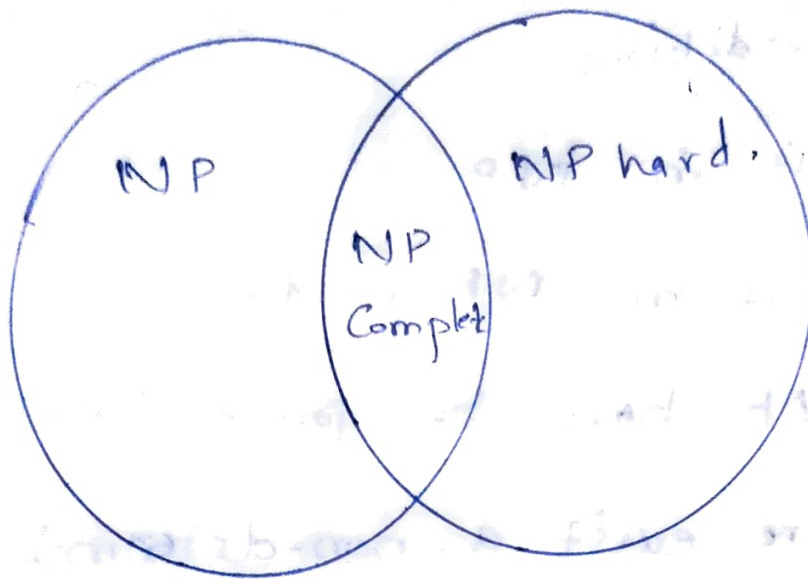
Problems that satisfies the second conditions of NP Completeness is said to be NP hard.

i.e., a language L is said to be in NP hard, if

$L' \leq_p L$, where $[L'$ is polynomial

time reducible to L , where

L is already proved to be in class NP Complete.



This is the most likely relationship b/w Complexity classes.

If any NP Complete problem can be solved in polynomial time, then every NP Complete problem has a polynomial time Algorithm.

Most theoretical Computer Scientists

believe that, the NP Complete problems are intractable.

All NP Complete problems are NP hard, but some NP hard problems are not known to be NP Complete.

For example, only a decision problem can be NP Complete.

However an optimization problem may be NP hard.

If L_1 is a decision problem and L_2 an optimization problem, it is quite possible that

$$L_1 \leq_p L_2.$$

One can also show that $L_2 \leq_p L_1$. But it is hard.

So optimization problems cannot be NP Complete but decision problems can.

Polynomial time Reduction

Let us consider a decision problem, A , which would like to solve in polynomial time.

We call the input to a particular problem, an instance of that problem.

Suppose we have a procedure that transforms any instance α of A into some instance β of B with the following characteristics:

1 $\Rightarrow$ The transformation takes polynomial time.

2 $\Rightarrow$ The answers are the same, i.e., the answer for α is 'yes' if and only if the answer for β is also 'yes'.

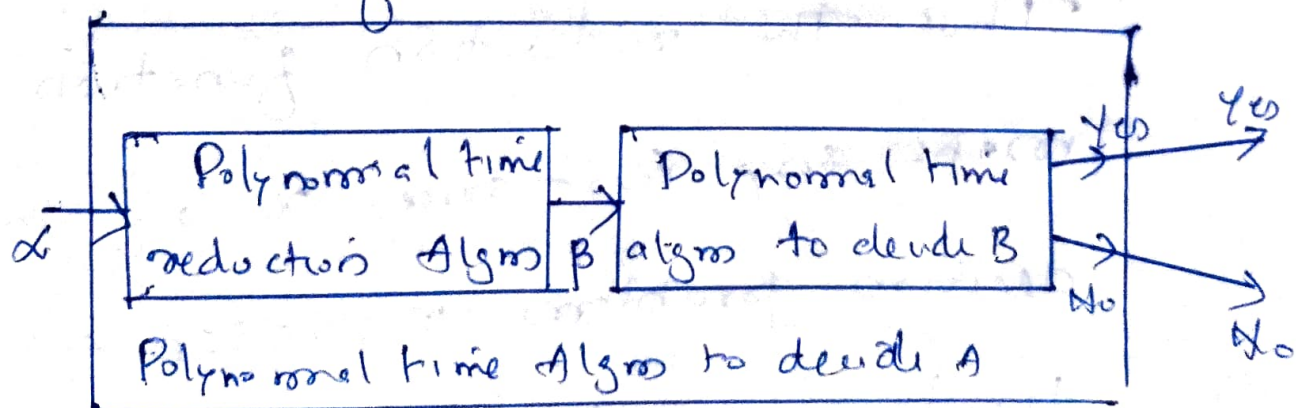
This type of procedure is known as "polynomial time reduction algorithm".

It provides us a way to solve problem A in polynomial time

① $\Rightarrow$ Given an instance α of problem A, use a polynomial time reduction algorithm to transform it into an instance β of problem B.

② $\Rightarrow$ Run the polynomial time decision algorithm for B on the instance β .

③ $\Rightarrow$ Use the answer for β as the answer for α .



In general, a language L_1 is a polynomial time reducible to language L_2 [$L_1 \leq_p L_2$] if there exist a polynomial time computable function " f " called as reduction function and a polynomial time algorithm A , that computes reduction function f .

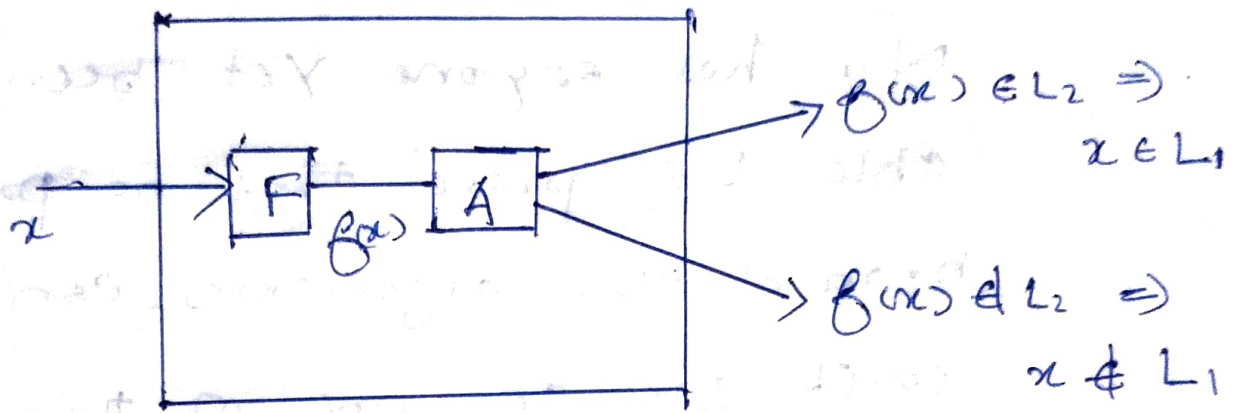
The reduction function f provides a polynomial time mapping such that if $x \in L_1$ then $f(x) \in L_2$.

Moreover if $x \notin L_1$ then $f(x) \notin L_2$.

Thus the reduction function maps any instance x of the decision problem represented by the language L_1 to an instance $f(x)$ of the problem represented by

L_2 .

Providing an answer to whether $\phi(x) \in L_2$ directly provides answer to whether $x \in L_1$.



$$L_1 \leq_p L_2$$

Polynomially Equivalent

Two problems L_1 and L_2 are said to be polynomially equivalent if and only if $L_1 \leq_p L_2$ and $L_2 \leq_p L_1$.

NOTE

No polynomial time algorithm has yet been discovered for an NP Complete problem.

Nor has anyone yet been able to prove that no polynomial time algorithm can exist for any one of them.

This is perhaps the most famous unsolved problem in computer science, i.e.,

Whether $P = NP$ or $P \neq NP$.

This has been one of the deepest, most perplexing open research problems in theoretical computer science since it was first posed in 1971.

First NP Complete Problem.

The problem, that we shall use as the first NP complete problem is Circuit Satisfiability.

It is known as Cook's Theorem.

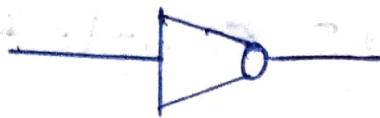
Circuit Satisfiability

Boolean combinational circuits are built from boolean combinational elements that are interconnected by wires.

The boolean combinational elements that we use in the circuit satisfiability problem compute a simple boolean function and they are known as logic gates.

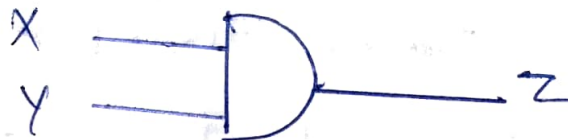
The 3 basic logic gates that we use in the circuit satisfiability problems are

⇒ NOT GATE



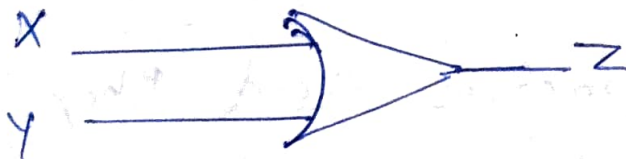
x	$\neg x$
0	1
1	0

⇒ AND GATE



x	y	z
0	0	0
0	1	0
1	0	0
1	1	1

⇒ OR GATE

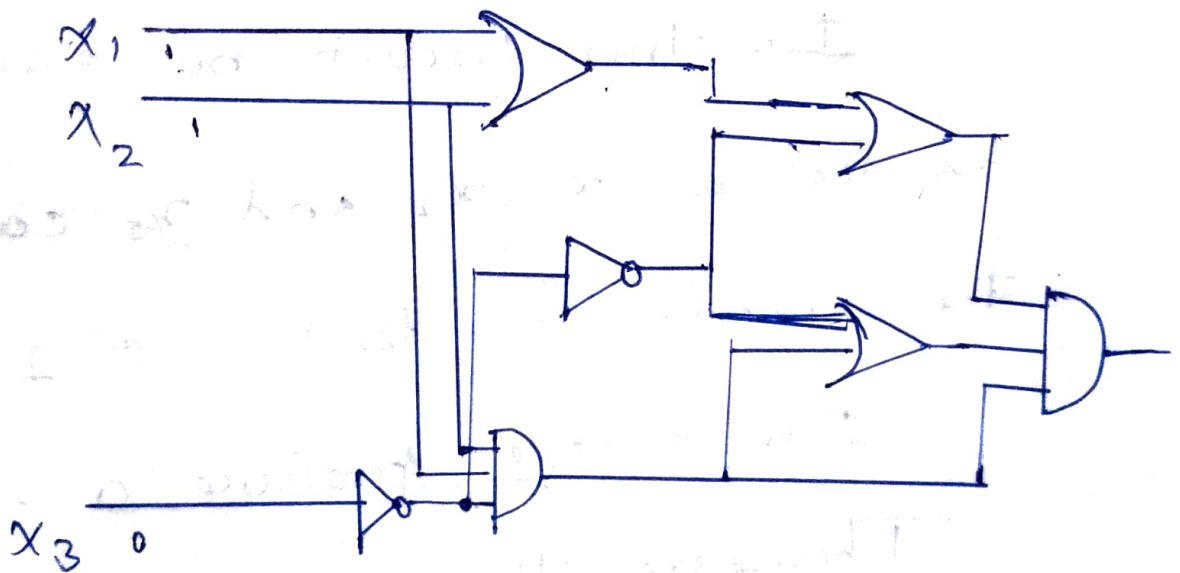


x	y	z
0	0	0
0	1	1
1	0	1
1	1	1

We can say a boolean combinational circuit is satisfiable if it has a satisfying assignment.

Satisfying assignment is a truth assignment that causes the output of the circuit to be 1.

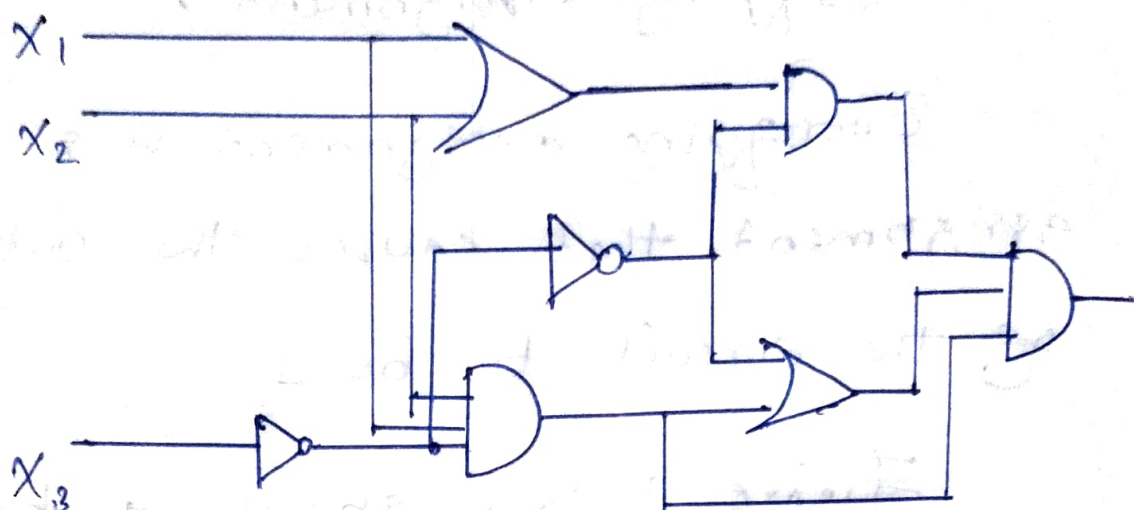
Figure below shows a boolean combinational circuit



This circuit is satisfiable, because it has a satisfying assignment.

$$x_1 = 1 ; \quad x_2 = 1 \text{ and } x_3 = 0$$

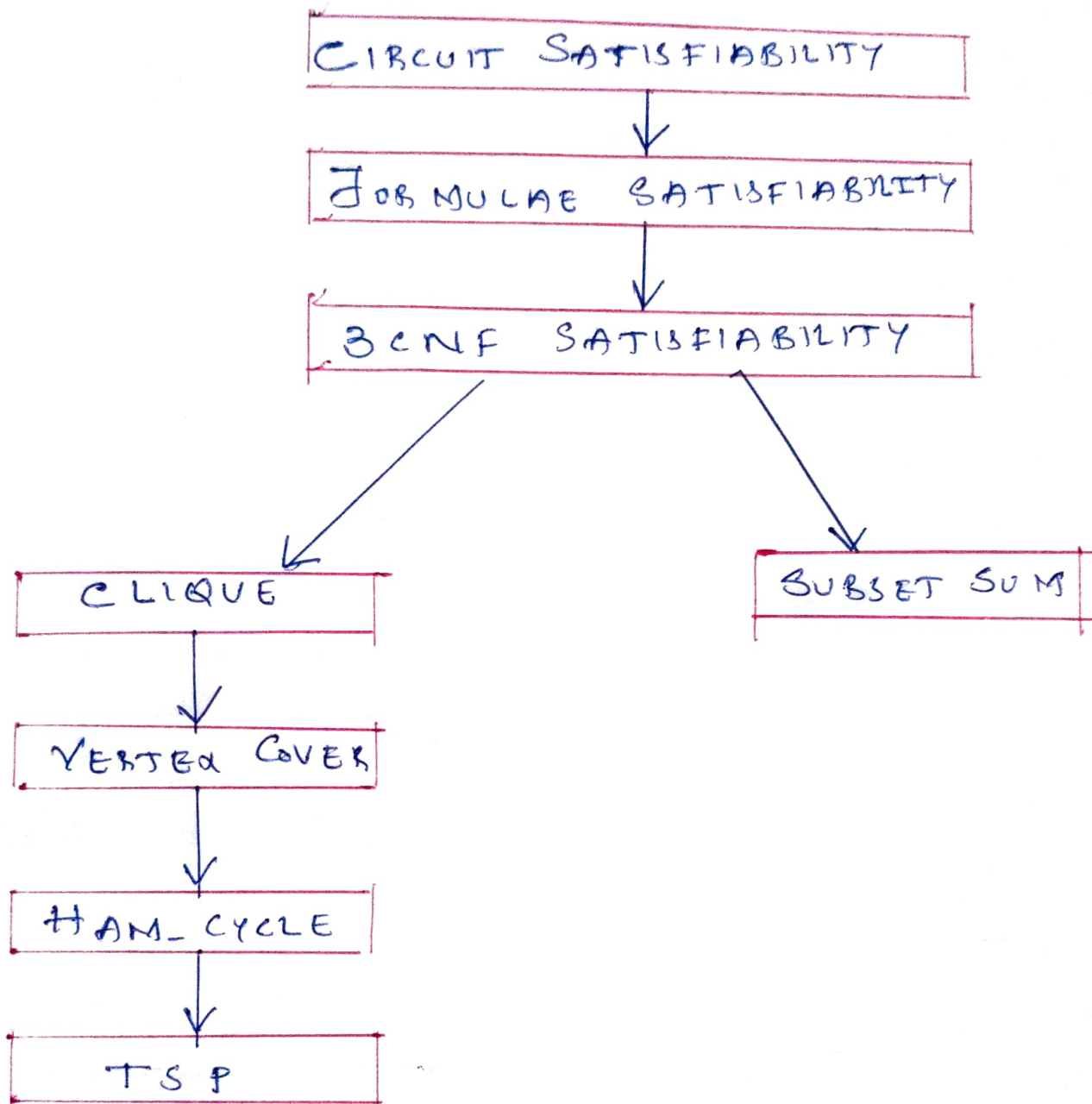
And now Consider another
combinational circuit given below



In this circuit no assignment
of values x_1 , x_2 and x_3 causes
the circuit to produce a 1 output.
It always produce 0.

Therefore this circuit is not
satisfiable.

WELL KNOWN NP COMPLETE PROBLEMS



FORMULAE SATISFIABILITY

SAT is a boolean formulae ϕ
composed of

1. n boolean variables ($x_1, x_2, \dots, x_n$)
2. m boolean connectives, such as
 - AND ($\wedge$)
 - OR ($\vee$)
 - NOT ($\neg$)
 - Implication ($\rightarrow$)
 - If and only if ($\leftrightarrow$)

and

③ Parenthesis

A truth assignment for a boolean formula is a set of values for the variable of ϕ .

And a satisfying assignment is a truth assignment that causes it to evaluate to 1.

For eg:- $\phi = (x_1 \vee x_2 \vee \neg x_3) \wedge (x_1 \vee \neg x_2)$

$$x_1 = 1 \quad x_2 = 0 \quad x_3 = 1$$

$$\phi = (1 \vee 0 \vee 0) \wedge (1 \vee 1)$$

$$= 1 \wedge 1$$

$$= 1$$

Theorem - 1

Satisfiability of boolean formulae is NP complete

Proof

We have to prove

1- SAT $\in$ NP

2- ~~NP~~ SAT $\in$ NP Hard

for that circuit SAT is polynomial time reducible to SAT.

SAT $\in$ NP

To show that SAT belongs to NP
 We show that a certificate consisting of a satisfying assignment for an input formula ϕ can be verified in polynomial time.

Verifying algorithm simply replaces each variable in the formula with its corresponding value and then evaluate the expressions.

This task is easily done in polynomial time.

$\therefore$ SAT $\in$ NP

SAT $\in$ NP-HARD

To prove that SAT is NP hard
 We have to show that ~~circuits~~

* CIRCUIT-SAT $\leq_p$ SAT *

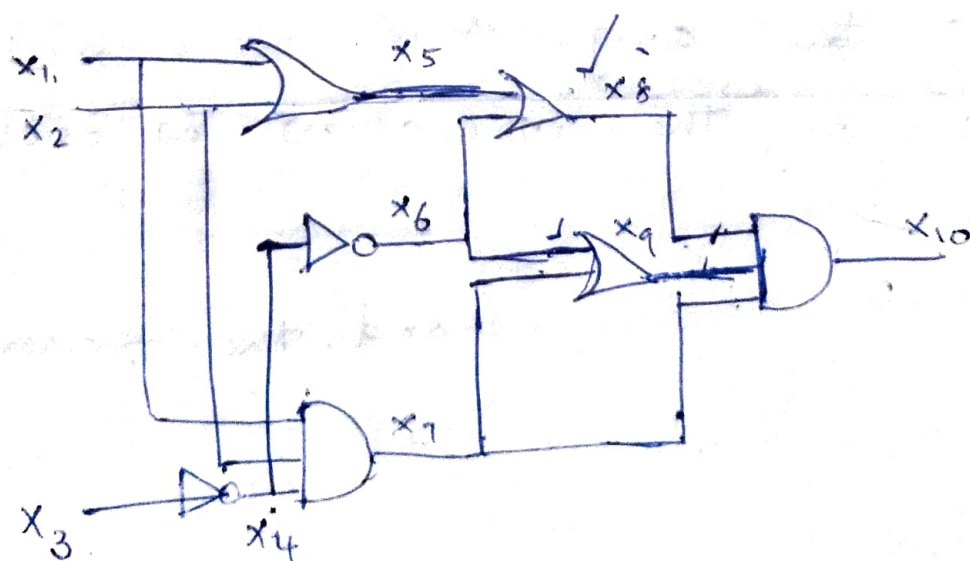
i.e., we have to show that, any instance of circuit satisfiability can be reduced in polynomial time to an instance of formulae satisfiability

29 → Induction can be used to express any combinational circuit as a boolean formula.

We simply look at the gate that produces the circuit output and inductively express each of the gate's input as a formula.

But this straight forward method doesnot constitute a polynomial time reduction.

For eg:- The SAT reduced from the circuit sat using straight-forward method for the below circuit will be like this.



$$\phi = ((x_1 \vee x_2) \vee \neg x_3) \wedge ((x_1 \wedge x_2 \wedge \neg x_3) \vee x_3) \wedge (x_1 \wedge x_2 \wedge \neg x_3).$$

20 This reduction will not take place in polynomial time.

There is another method of reduction from Circuit SAT to SAT.

In this method, for each wire x_i in the circuit C , the formulae ϕ has a variable x_i .

The proper operation of a gate can now be expressed as a formula involving the variables of its incident wires.

The formulae ϕ produced by the reduction algorithm is the AND of the circuit output variable with the conjunction of clauses describing the operation of each gate.

By this method, the generated boolean formula SAT for the above circuit will look like this:-

$$\begin{aligned}
 \phi = & x_{10} \wedge (x_0 \leftrightarrow \neg x_3) \\
 & \wedge (x_5 \leftrightarrow (x_1 \vee x_2)) \\
 & \wedge (x_6 \leftrightarrow \neg x_4) \\
 & \wedge (x_7 \leftrightarrow (x_1 \wedge x_2 \wedge x_0)) \\
 & \wedge (x_8 \leftrightarrow (x_5 \vee x_6)) \\
 & \wedge (x_9 \leftrightarrow (x_6 \vee x_7)) \\
 & \wedge (x_{10} \leftrightarrow (x_7 \wedge x_8 \wedge x_9))
 \end{aligned}$$

This reduction can be done in polynomial time.

claim-1

Circuit is satisfiable $\Rightarrow$ SAT is satisfiable.
--

When a circuit is given, it is easy to produce a formula ϕ in polynomial time.

If C has a satisfying assignment each wire of the circuit has a well defined value and the output of the circuit is 1.

Therefore the assignment of wire values to variables in ϕ makes each clause of ϕ evaluates to 1 and thus conjunction of all evaluates to 1.

claim 2

$\text{SAT is satisfiable} \Rightarrow \text{Circuit SAT is satisfiable.}$
--

If there is an assignment that causes ϕ to evaluate to 1, then the circuit C will be satisfiable.

From claim ① and ② SAT is NP hard

Since SAT is NP and NP hard it is NP Complete.

3CNF Satisfiability

We can define 3CNF satisfiability using the following terms.

Literal

A variable or its negation

CNF $\rightarrow$ Conjunctive Normal Form

A boolean formula is in CNF if it is expressed as an AND of clauses, each of which is the OR of one or more literals.

23

3CNF

A boolean formula is in 3CNF if each clause has exactly 3 distinct literals.

eg:- $(x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_3)$

Theorem

Satisfiability of boolean formulas in 3CNF is NP Complete.

Proof

We have to prove

1- 3SAT $\in$ NP

② SAT $\leq_p$ 3SAT

1- 3CNF SAT $\in$ NP.

We already proved that ~~3SAT~~ SAT is NP. Therefore 3CNF SAT is also in NP.

2. 3CNF $\in$ NP-Hard

In order to prove 3CNF is NP-hard we have to show that

SAT $\leq_p$ 3CNF SAT.

Let C_i be the clause in boolean formula and ϕ be the 3CNF

We can reduce C_i into ϕ

using the following steps.

1- If a clause C_i has 3 distinct literals - make the clause as it is

2- If C_i has 2 distinct literals

$C_i = (l_1 \vee l_2)$, add one extra variable p as shown

$$(l_1 \vee l_2 \vee p) \wedge (l_1 \vee l_2 \vee \neg p).$$

Let it will be clauses of ϕ .

3- If C_i has only 1 literal include 2 variables p and q such as

$$(l_1 \vee p \vee q) \wedge (l_1 \vee p \vee \neg q) \wedge$$

$$(l_1 \vee \neg p \vee \neg q) \wedge (l_1 \vee \neg p \vee q)$$

as the clauses of ϕ .

4- If C_i has more than 3 literals

$C_i = (l_1 \vee l_2 \vee \dots \vee l_k)$ add a new variable p , such that

$$C_i = (l_1 \vee l_2 \vee p) \wedge (\neg p \vee l_3 \vee l_4) \wedge \dots$$

This reduction can be done in Polynomial time.

So BCNF SAT is NP Hard

Since BCNF SAT is in NP and NP Hard we can say that BCNF SAT is NP Complete.

39
25 COPY

CLIQUE

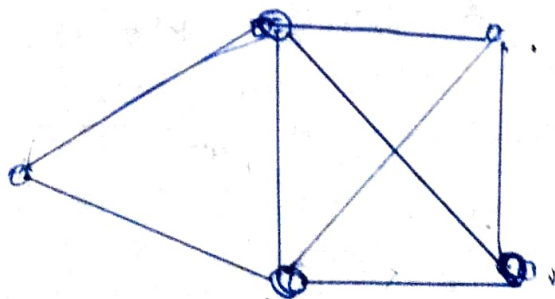
A clique in an undirected graph $G = (V, E)$ is a subset $[V' \subseteq V]$ of vertices, in which each pair of which is connected by an edge E .

In other words a clique C of graph G is a complete subgraph of G . i.e., every two nodes of C are adjacent.

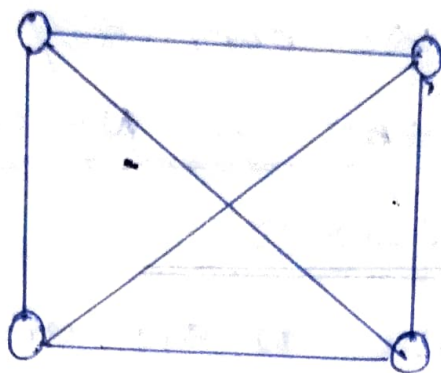
The size of a clique is the number of vertices it contains.

The clique problem is the optimization problem of finding a clique of maximum size in a graph.

Eg:-



From this graph we can form a K_4 -clique



In this all the nodes are connected each other.

Theorem

The clique problem is NP-Complete

Proof

We have to prove

1- clique $\in$ NP, and

2- $\exists$ CNF-SAT $\leq_p$ clique.

clique $\in$ NP

To show that clique $\in$ NP, for a graph $G = (V, E)$, we use the set of vertices $V' \subseteq V$ in the clique as a certificate, for G .

checking whether V' is a clique can be accomplished in polynomial time by checking whether for each pair $u, v \in V'$ the edge (u, v) belongs to E .

clique $\in$ NP hard

For this you have to prove that

3CNF-SAT $\leq_p$ clique.

The reduction algorithm begins with an instance of 3CNF SAT

Let $\phi = c_1 \wedge c_2 \wedge \dots \wedge c_k$ be a boolean formula in 3CNF with k -clauses

In which each clause c_i has 3 literals

$$c_i = l_1^r \vee l_2^r \vee l_3^r$$

We can construct a graph G such that ϕ is satisfiable iff G has a clique of size k .

The graph $G(V, E)$ is constructed as follows:-

① For each clause $c_i = l_1^r \vee l_2^r \vee l_3^r$ in ϕ we place a triple of vertices

28 v_1^r, v_2^r and v_3^r in V .

② We put an edge between two vertices if both of the following hold

- ① No edge between two literals in the same clause.
- ② No edge b/w complement any literal.

This graph can easily be computed from ϕ in polynomial time.

claim-1

ϕ is satisfiable $\Rightarrow$ Clique is satisfiable.

Suppose that ϕ has a satisfying assignment.

Then each clause C_r contains at least one literal l^r that is assigned 1 and each such literal

corresponds to a vertex V_i^v .

Picking one such true literal from each clause yields a set V' of k -vertices.

We claim that V' is a clique because:-

For any two vertices $V_i^v, V_j^s \in V'$ both corresponding literals l_i^v and l_j^s are mapped to 1 by the given satisfying assignment and thus the literals cannot be complements.

Thus by the construction of G , the edge (V_i^v, V_j^s) belongs to E .

claim-2

G has a clique of size $k \Rightarrow \phi$ is satisfiable.

Suppose that G has a clique V' of size k .

No edge in G connects vertices in the same tuple and so V' contains exactly one vertex per tuple.

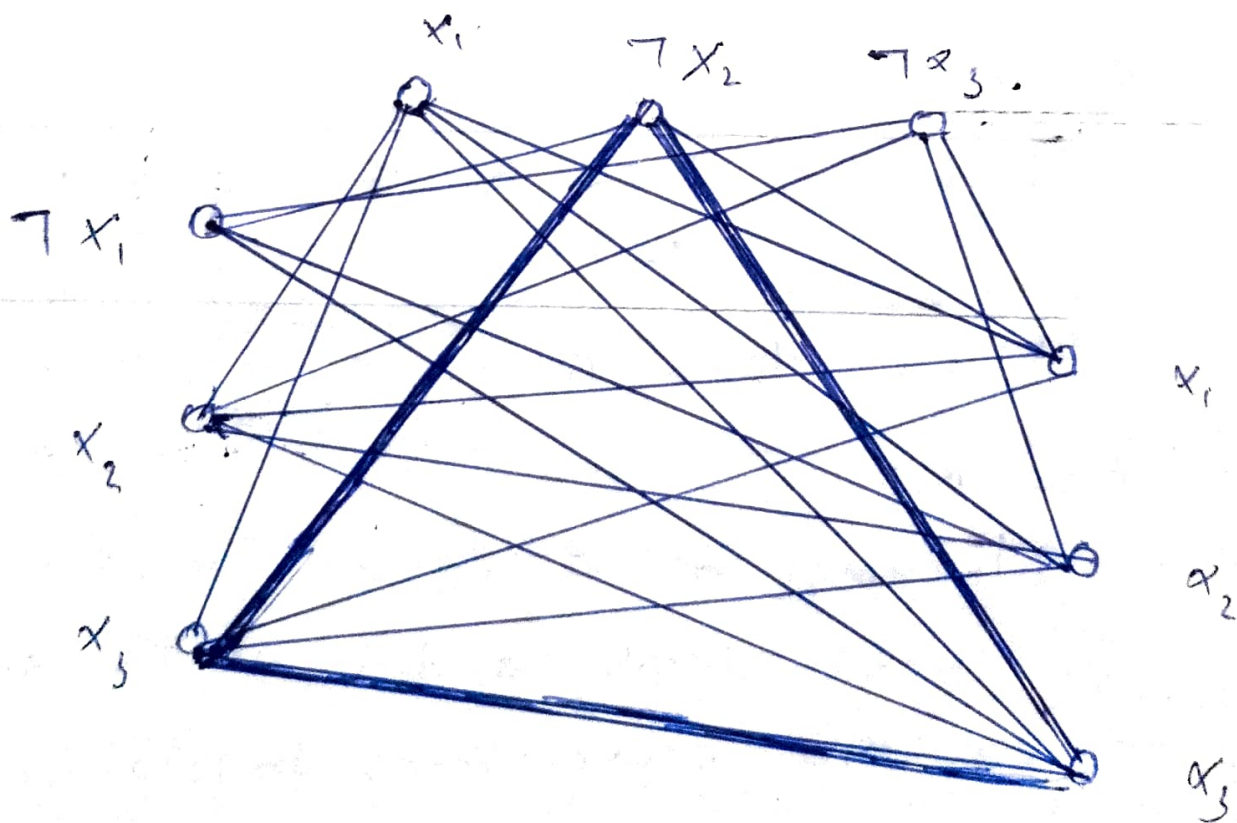
We can assign 1 to each literal L_i s.t. $V_i \in V'$ without fear of assigning 1 to both L_i and its complement.

Each clause is satisfied and so ϕ is satisfied

Eg:- 18 We have

$$\phi = (x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3)$$

then G is the graph shown below



The graph G is derived from the 3 CNF formula $\phi = c_1 \wedge c_2 \wedge c_3$, where

$$c_1 = (x_1 \vee \neg x_2 \vee \neg x_3)$$

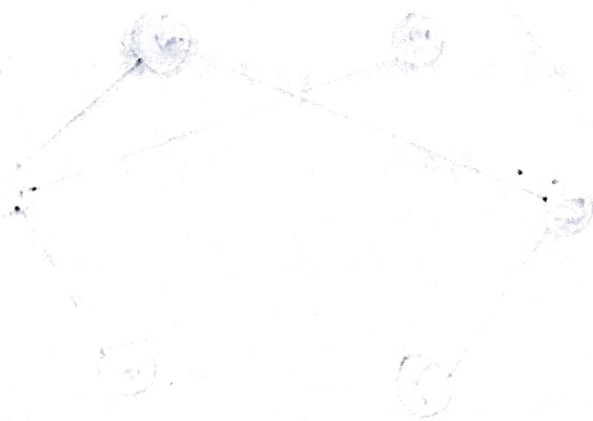
$$c_2 = (\neg x_1 \vee x_2 \vee x_3) \text{ and}$$

$$c_3 = (x_1 \vee x_2 \vee x_3) \text{ is reducing 3CNF set to clique.}$$

A satisfying assignment of the formula has

$$x_1 = 0 \text{ or } 1 \quad x_2 = 0 \quad x_3 = 1$$

This assignment satisfies c_1 with $\neg x_2$ and it satisfies c_2 and c_3 with x_3 corresponding to the clique with thickly shaded edges.



VERTEX COVER

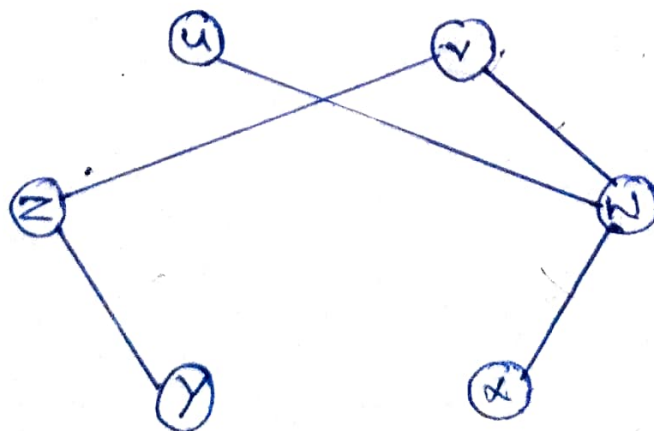
A vertex cover of an undirected graph $G = (V, E)$ is a subset $V' \subseteq V$ such that if $(u, v) \in E$ then $u \in V'$ or $v \in V'$ or both.

That is, a vertex cover for G is a set of vertices that covers all the edges in E .

The size of a vertex cover is the number of vertices in it.

The vertex cover problem is to find a vertex cover of minimum size in a given graph.

For example the graph in figure shown below has a vertex cover $\{w, z\}$ of size 2.



Theorem

The vertex cover problem is NP Complete

Proof

In order to prove vertex cover problem is NP Complete we have to show that

① Vertex Cover $\in$ NP

② Vertex Cover $\in$ NP Hard.

Vertex Cover $\in$ NP

Suppose we are given a graph $G=(V,E)$ and an integer k .

The certificate we choose is the vertex cover $V' \subseteq V$ itself.

The verification algorithm verifies that $|V'| \leq k$ and then it checks for each edge $(u,v) \in E$ that $u \in V'$ or $v \in V'$.

This verification can be performed in polynomial time.

Vertex Cover $\in$ NP Hard

Vertex Cover $\in$ NP Hard

To prove Vertex Cover is NP hard we have to show that

34

Clique $\leq_p$ Vertex Cover

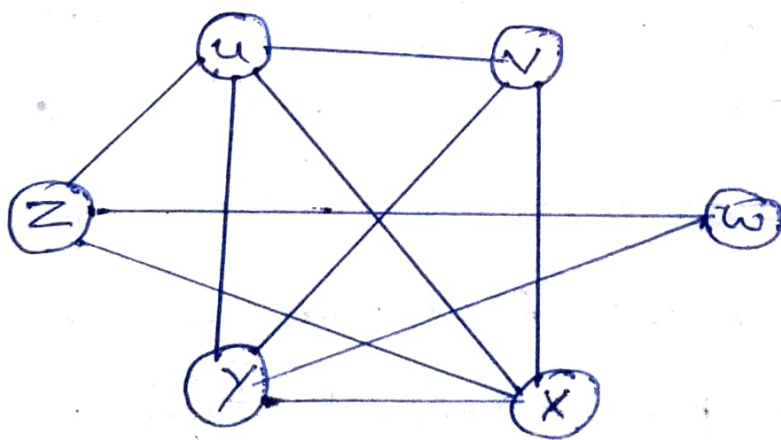
This reduction is based on the complement of a graph.

Given an undirected graph

$G = (V, E)$ We define the complement of G as $\bar{G} = (V, \bar{E})$, where $\bar{E} = \{ (u, v) \mid u, v \in V, u \neq v \text{ and } (u, v) \notin E \}$.

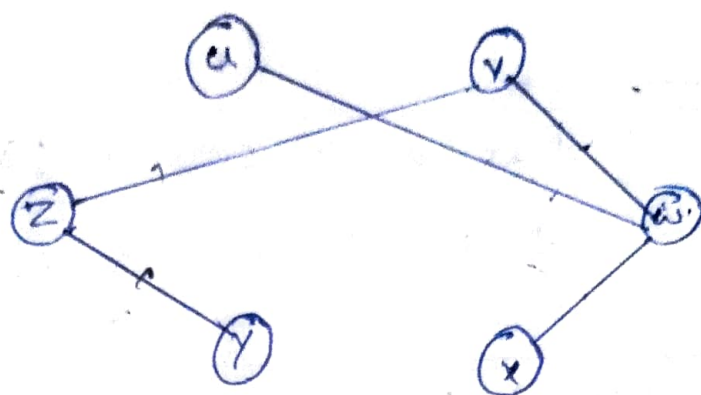
In other words $\bar{G}$ is the graph containing exactly those edges that are not in G .

For example, Figure below shows a graph



Here the clique is (u, v, x, y)

Complement of this graph is shown below:-



The vertex cover of this graph is $\{w, z\}$, that is the complement of the ~~graph~~ ~~clique~~ ~~clique~~ where clique $\{u, v, x, y\}$.

The reduction's algorithm takes as input an instance $\langle G, k \rangle$ of the clique problem.

It computes the complement $\bar{G}$.

The output of the reduction's algorithm is the instance $\langle \bar{G}, |V| - k \rangle$ of the vertex cover problem.

To complete the proof we have to show that this transformation is a reduction.

ie, we have to show that the graph G has a clique of size k if and only if the graph $\bar{G}$ has a vertex cover of size $|V| - k$.

36

claim-1

If G has a clique $V' \subseteq V$ with $|V'| = k$
 $\Rightarrow V - V'$ is a vertex cover in G .

Let (u, v) be any edge in E . then

$$(u, v) \neq \emptyset$$

$\Rightarrow$ at least one of u or v does not belong to V' .

Equivalently at least one of u or v is in $V - V'$ which means that edge (u, v) is covered by $V - V'$.

Since (u, v) was chosen arbitrarily from E every edge of E is covered by a vertex in $V - V'$.

Hence the set $V - V'$ which has size $|V| - k$ forms a vertex cover for G .

claim-2

Suppose that G has a vertex cover $V' \subseteq V$ where $|V'| = |V| - k$.

Then $\forall u, v \in V$ if $(u, v) \in E$

then $u \in V'$ or $v \in V'$ or both.

The contrapositive of this implication is that $\forall u, v \in V$ if $u \notin V'$ and $v \notin V'$ then $(u, v) \in E$.

In other words $V - V'$ is a clique and it has size $|V| - |V'| = k$.

Approximation Algorithms

The NP Complete problems are likely to be quite hard to solve exactly. There are mainly three approaches to solve an NP Complete problem.

→ Try exponential brute algor.

→ Try General Optimization methods

→ Try Approximate Algorithms.

Among these approaches one approach is the use of fast

59 algorithms that are not guaranteed to give best solution but will give one that is close to optimal.

Such algorithms are called Approximation algorithms or heuristic algorithms.

For example

⇒ For the travelling Salesman problem, the optimization problem is to find the shortest cycle; and the approximation problem is to find the short cycle.

⇒ For the vertex Cover problem the optimization problem is to find the vertex cover with fewest vertices, and the approximation problem is to find the vertex cover with few vertices.

The goal of an approximation is to come as close to the optimum value as possible in reasonable amount of time.

Approximation Algorithm for Vertex Cover

Algorithm

Approx - Vertex - Cover (G)

$C \leftarrow \emptyset$

$E' \leftarrow E(G)$

While $E' \neq \emptyset$

do let (u, v) be an arbitrary edge of E'

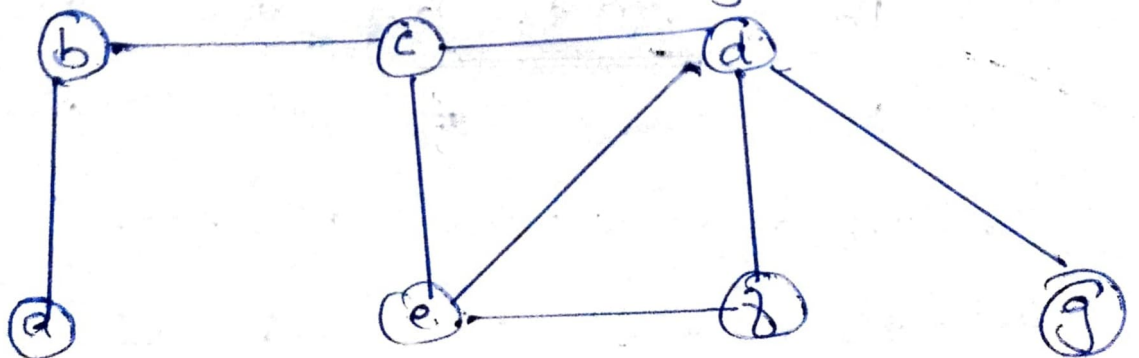
$C \leftarrow C \cup \{u, v\}$

remove from E' every edge incident on either u or v

return C .

Example

- ① Input graph G which has 7 vertices and 8 edges.



6) $C \leftarrow \phi$

$E' = E(G) = \{ (b,c), (c,d), (d,e), (e,f), (f,g) \}$

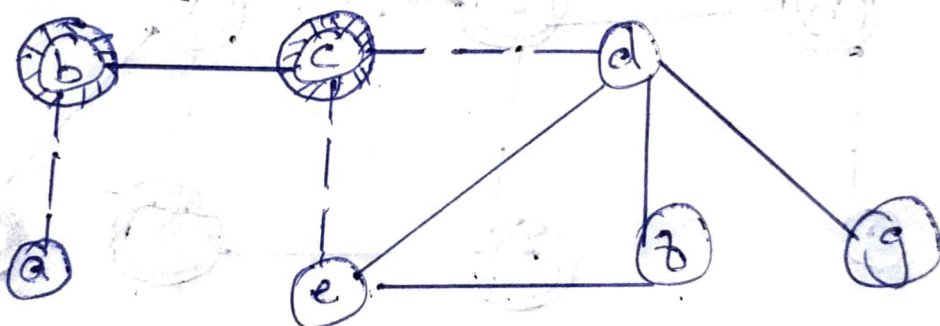
While $E' \neq \phi$

$\Rightarrow$ Let edge (b,c) is the first edge chosen by the algorithm and added to the set C .

$C = \{ (b,c) \}$

$\Rightarrow$ Remove from E' every edge that is incident on either 'b' or 'c'

$E' = \{ (b,c), (c,d), (e,f), (f,g) \}$



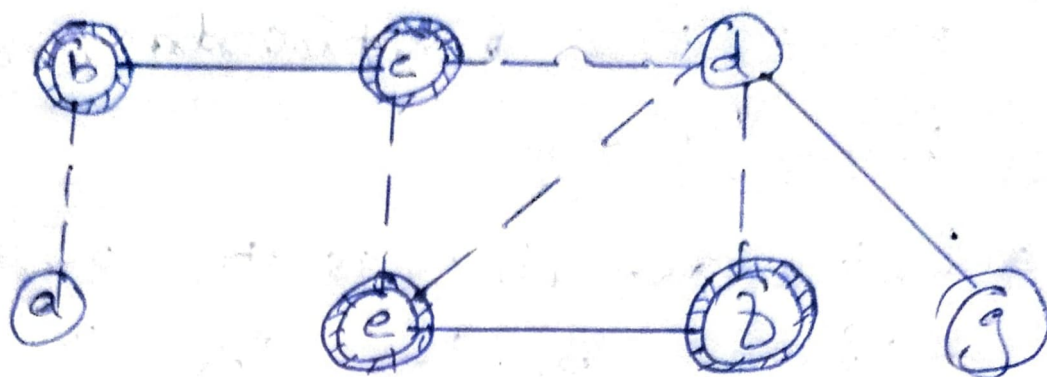
$\Rightarrow$ Let edge (e,f) is chosen next and added to set C .

$C = \{ (b,c), (e,f) \}$

$\Rightarrow$ Remove from E' every edge that is incident on either 'e' or 'f'.

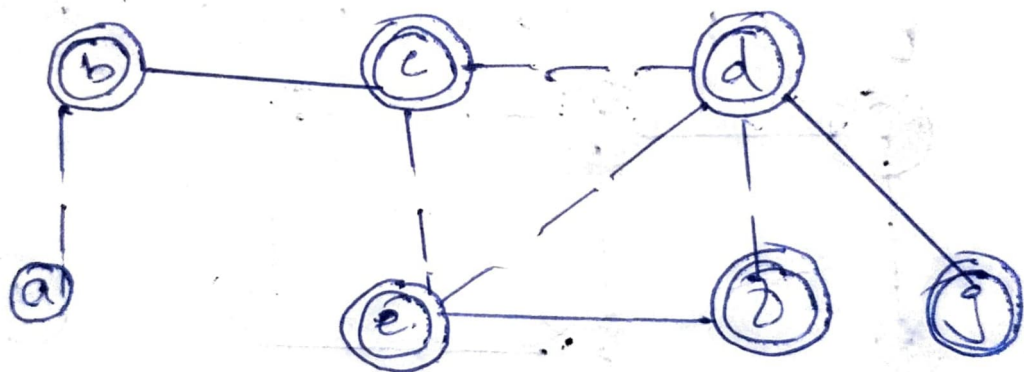
b2

$$E' = \{(b, c), (c, d), (d, g)\}$$



⇒ Next edge (d, g) is chosen and added to c .

⇒ Remove all edges that is incident on either d or g .



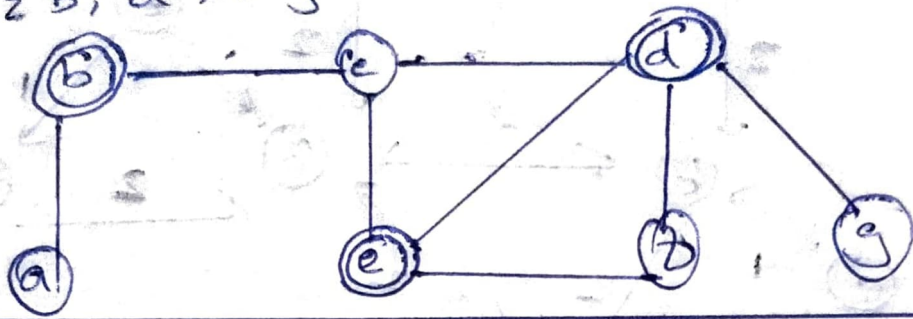
$$C = \{b, c, e, f, d, g\}$$

⇒ The set C is the vertex cover produced by the algorithm and it contains 6 vertices $\{b, c, d, e, f, g\}$.

Note:-

The optimal vertex cover for this problem, contains only three vertices

$\{b, d, e\}$.



Bin Packing

In bin Packing problem we are given a set of n objects, where the size s_i of the i th object satisfies $0 < s_i < 1$.

We wish to pack all the objects into the minimum number of unit size bins.

Each bin can hold any subset of the objects whose total size does not exceed 1.

There are two versions of bin packing algorithms available.

⇒ Online

items arrive one at a time, each must be put in a bin, before considering the next item.

⇒ Offline

All items given upfront.

Online Algorithms

- Next Fit
- Best Fit
- First Fit
- Worst Fit

Next Fit

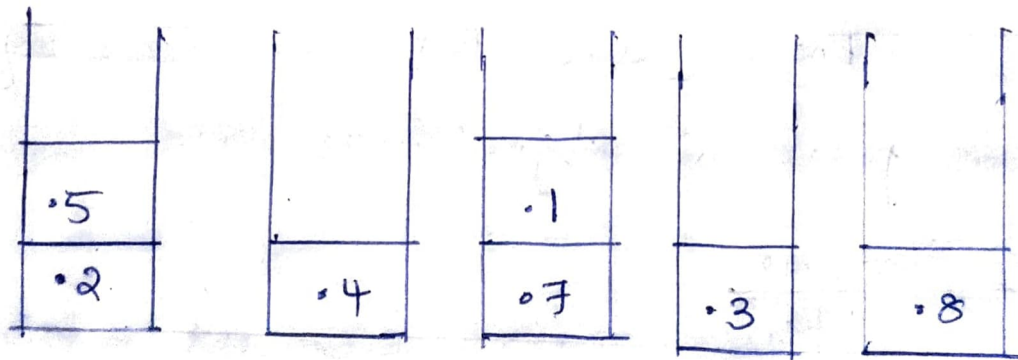
Place the items in the order in which they arrive.

Place the next item into the current bin if it fits.

If it does not, close that bin and start a new bin.

Example

.2, .5, .4, .7, .1, .3, .8.



Total No. of bins = 5

First Fit

Place the items in the order in which they arrive.

Place the next item into the lowest numbered bin in which it

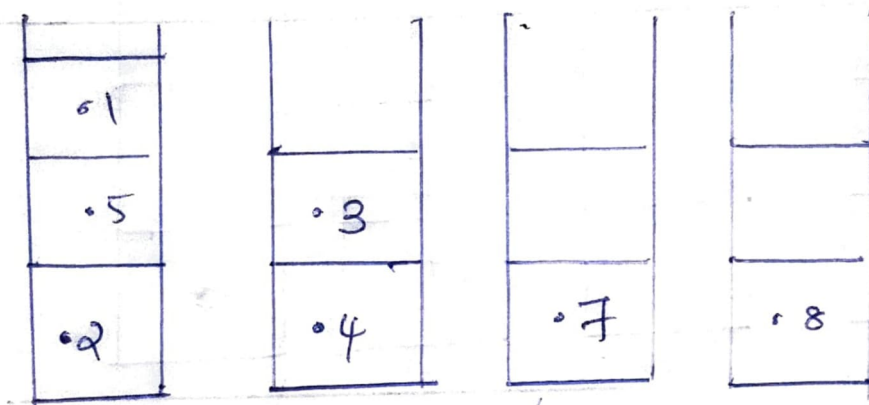
fit

i.e., We check all previous bins to see if the next item will fit.

If it does not fit into any open bin, start a new bin.

Example

2, 5, 4, 7, 1, 3, 8.



Total No. of bins = 4

Best Fit

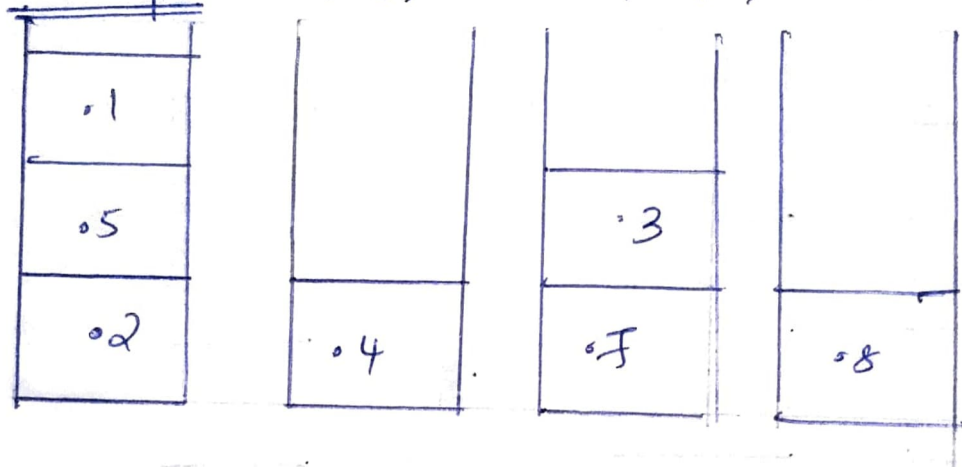
Place the items in the order in which they arrive.

Place the next item into the bin which will leave the least room left over after the item is placed in the bin.

i.e., Best fit strategy places the next item in the tightest spot. That is, put it in the bin so that smallest empty space is left. If it does not fit in any bin

Start a new bin.

Example: 2, 5, 4, 7, 3, 8



Total No. of bins = 4

Worst Fit

- Place the items in the order in which they arrive.
- Place the next item into that bin which will leave the most room left over after the item

is placed in the bin.

If it doesnot fit in any bin,
start a new bin.

Example

• 2, • 5, • 4, • 7, • 1, • 3, • 8

	• 3		
• 5	• 1		
• 2	• 4	• 7	• 8

Total No. of Bins = 4.

Offline Algorithms

First Fit Decreasing

- Sort the items in decreasing order.

- Place the next item into that bin the lowest numbered bin in which it fits.

- If it doesnot fit into any open bin, start a new bin.

Graphs

Example

2, 5, 4, 7, 1, 3, 8

⇒ First We Have to Sort the elements in decreasing order

⇒ 8, 7, 5, 4, 3, 2, 1

2	3	1
8	7	4
		5

Total No. of bins ≥ 3

This is the optimal one.

Complexity of Bin Packing Algorithms

Next Fit

$$\Rightarrow \text{Best Case} = O(n)$$

$$\Rightarrow \text{Average Case} = O(n)$$

$$\Rightarrow \text{Worst Case} = O(n)$$

First Fit, Best Fit, Worst Fit

First fit decreasing, Best fit decreasing

$$\Rightarrow \text{Best Case} = O(n \log n)$$

$$\Rightarrow \text{Average Case} = O(n^2)$$

$$\Rightarrow \text{Worst Case} = O(n^2)$$

Minimum No. of Bins Required

The lower bound on minimum number of bins required can be given as

$$\text{Min No. of Bins} = \left\lceil \frac{\text{Total Weight}}{\text{Bin Capacity}} \right\rceil$$

Approximation Ratio

The quality of an approximation algorithm is usually measured as the approximation ratio.

It is the ratio, between the value of the approximate solution and the value of the optimal solution.

$$\text{Approximation Ratio} = \frac{\text{Approximate Solution}}{\text{Optimal Solution}}$$

⇒ For minimization problems, approximation ratio is in $[1, \infty)$.

⇒ For maximization problems it is in $[0, 1]$.

Apptn

Different types of Graph Coloring

⇒ Vertex Coloring

⇒ Edge Coloring

⇒ Face Coloring

Vertex Coloring

⇒ Assignment of colors to vertices in a graph such that no two adjacent vertices share the same color.

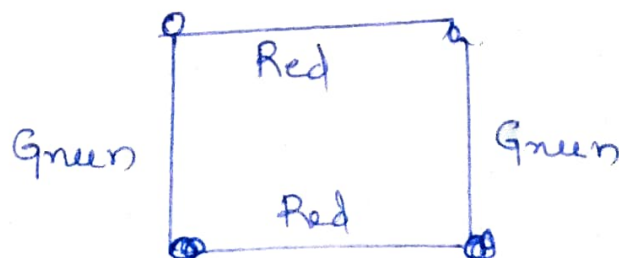
⇒ A graph is 0-colorable iff
 $V = \phi$

⇒ A graph is 1-colorable iff
 $E = \phi$

Edge Coloring

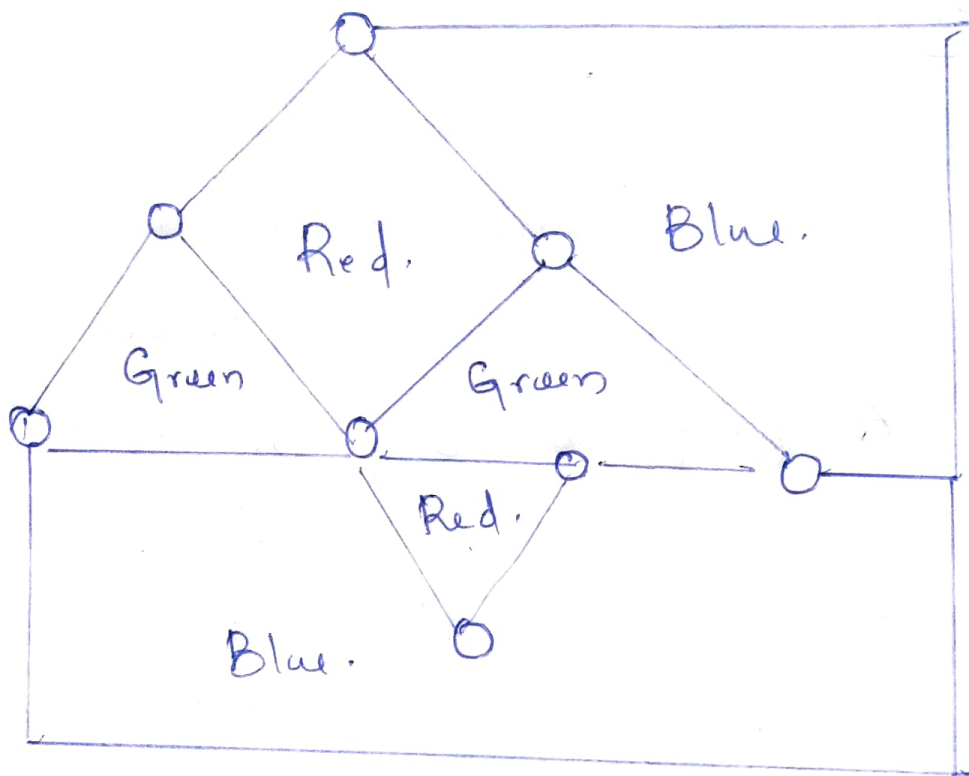
⇒ Given a graph $G = (V, E)$, assign a color to each edge so that no two adjacent edges share the same color.

Eg:-



Face Coloring

⇒ For a planer graph, assign a color to each face/region so that no two faces that shares boundary have the same color.



Approximation Algos for Graph Coloring

Approx - Graph - Coloring (G, n)

{ for $i \leftarrow 1$ to n

for $c \leftarrow 1$ to n

if no vertex adjacent to v_i
has color c

{

Color v_i with c

break

}

}

APPROXIMATION ALGORITHM FOR GRAPH COLORING

1. Color the first vertex with first color.
2. Do the following steps for remaining $n-1$ vertices.
3. Color the currently picked vertex v with the lowest numbered color that has not been used ~~on~~ previously by the

adjacent vertices

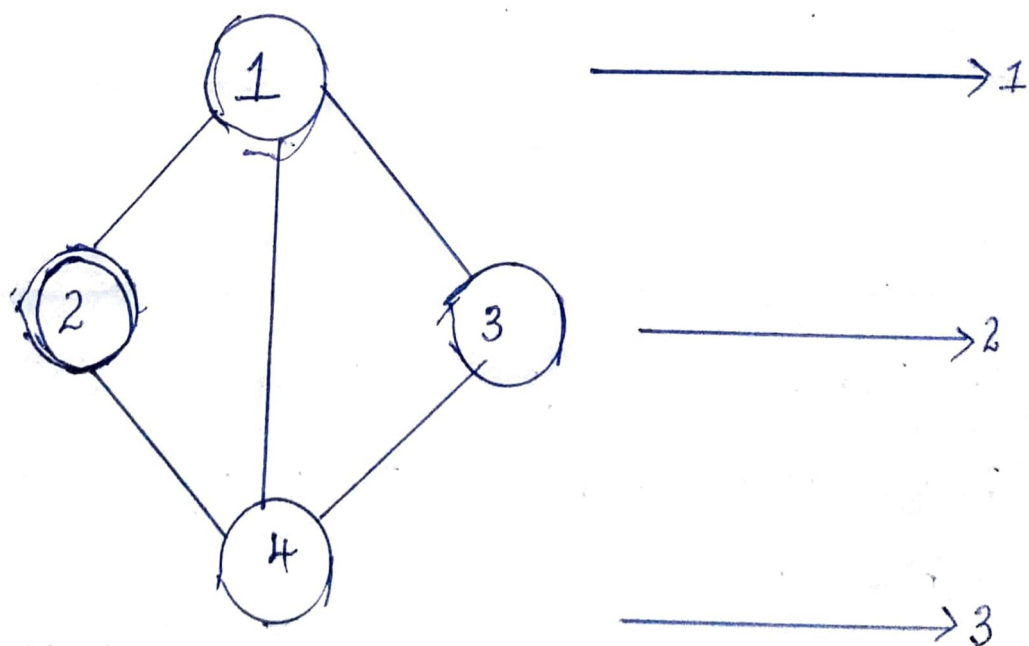
~~unlabeled previously used~~

4. If all lowest numbered colors are previously used, then assign a new color to it.

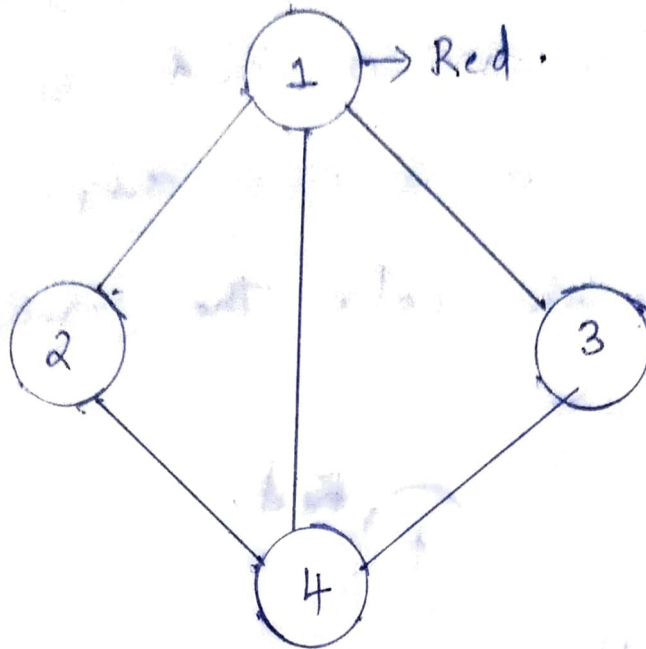
→ Here the order in which the vertices are picked is important

Welsh Powell Algm Considers vertices in descending order of degrees.

Example.

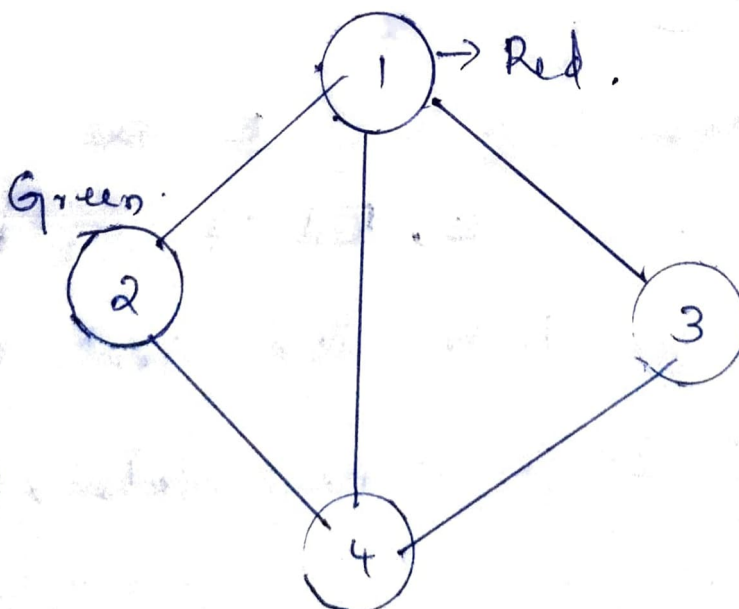


⇒ Color the node 1 with first color.
Say, Red.



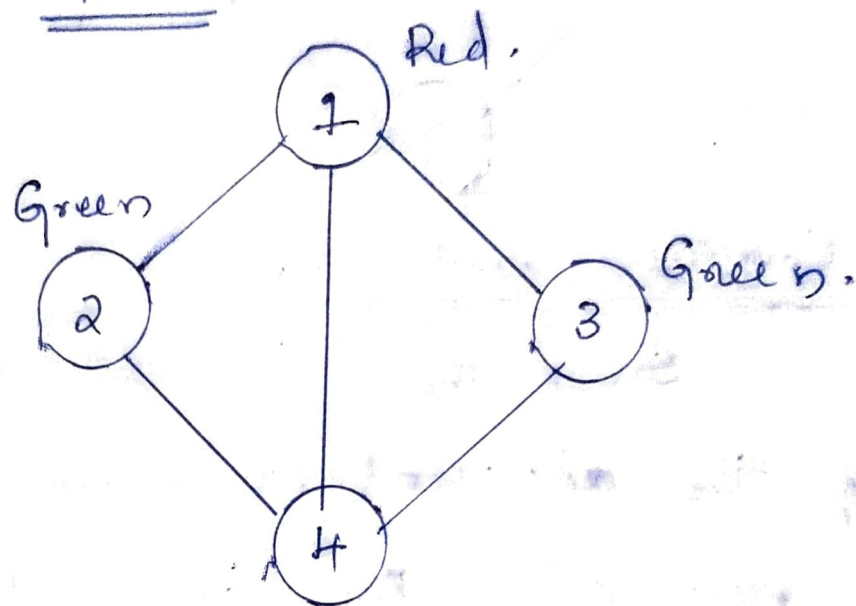
⇒ Consider node 2,

Since the color Red is used
by the adjacent node 1, we assign
a new color, Say Green to 2.



⇒ Consider node 3

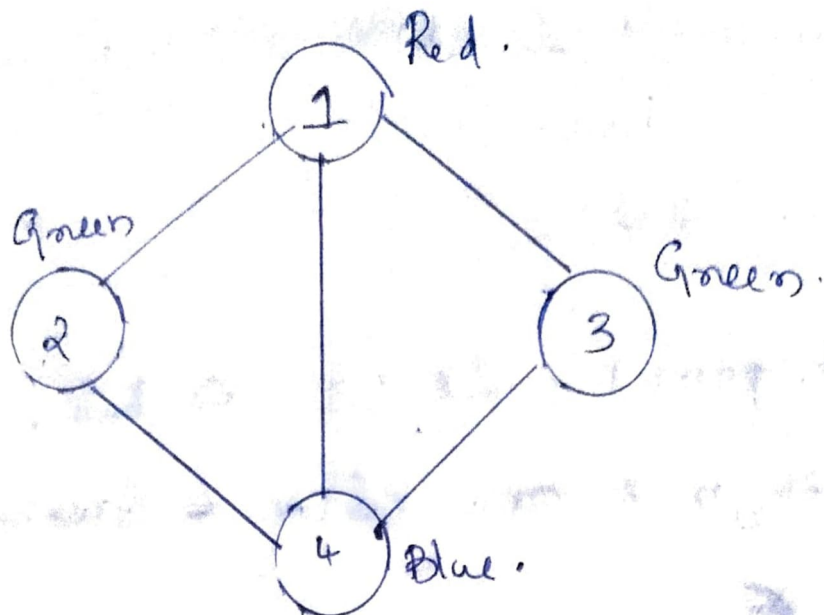
Here, the node adjacent to 3 is colored as Red and the color Green is not adjacent to node 3, We color the node 3 by the Green



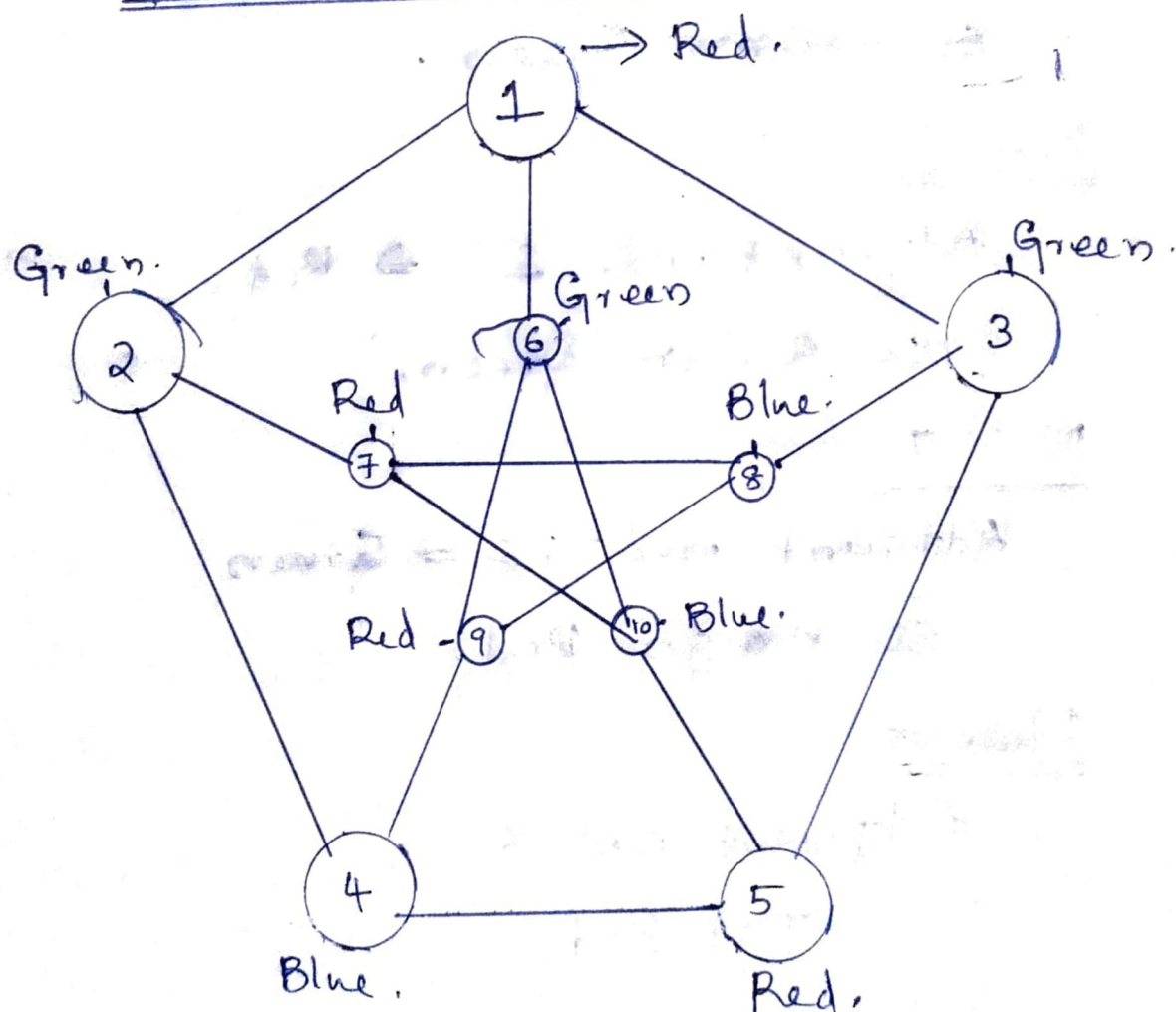
⇒ Consider node 4

Node 1, 2 and 3 are adjacent to the node 4, ~~Red~~ it can't be colored by both Red and Green.

So, assign a new color, Blue to it.



Total No. of colors used $\Rightarrow 3$



Total No. of colors used $\Rightarrow 3$

⇒ Color node 1 with first & color Red.

⇒ Node 2 ⇒

Adjacent node : 1 ⇒ Red.

So Assign a new color ⇒ Green

Node 5

Adjacent node : 1 ⇒ Red

So assign Green.

Node 3

Adjacent node : 1 ⇒ Red.

So assign Green.

Node 7

Adjacent node : 2 ⇒ Green

So assign Red.

Node 8

Adjacent nodes

7 ⇒ Red

3 ⇒ Green.

So Assign new color Blue.

node 9

Adjacent nodes

6 $\Rightarrow$ Green

8 $\Rightarrow$ Blue

So Assign Red.

node 10

Adjacent nodes

6 $\Rightarrow$ Green

7 $\Rightarrow$ Red

So Assign Blue

node 4

Adjacent nodes

2 $\Rightarrow$ Green

9 $\Rightarrow$ Red.

So Assign Blue.

node 5

Adjacent nodes

3 $\Rightarrow$ Green

10 $\Rightarrow$ Blue

So Assign Red.

Graph Coloring - Applications

1. Making Schedule or Timetable.

⇒ Suppose we want to make an exam schedule for a university

⇒ We have list of different subjects and students enrolled in every subject

⇒ Many subjects would have common students.

⇒ How do we schedule the exams so that no two exams with a common student are scheduled at same time?

⇒ How many min. time slots are needed to schedule all exams.

⇒ This problem can be represented as a graph where every vertex is a subject and an edge b/w two vertices means there is a common student.

⇒ So this is a graph coloring

problem where minimum no. of transmission slots is equal to the chromatic no. of the graph.

2. Mobile Radio Frequency Assignment.

⇒ When frequencies are assigned to towers, frequencies assigned to all towers at the same location must be different.

⇒ This problem is also an instance of graph coloring problem where every tower represents a vertex and an edge b/w two towers represents that they are in range of each other.

A randomized algorithm or

probabilistic algorithm is an algorithm which employs a degree of randomness as part of its logic.

This algorithm typically uses uniform random numbers as an auxiliary input to guide its behaviour in the hope of achieving good performance in the average case.

The main characteristic of probabilistic algorithms is that the same algorithm may behave differently when it is applied twice to the same instance.

Its execution time and even the result obtained may vary considerably from one run to the next.

Deterministic algorithm is an algorithm whose behaviour can be completely predicted from the input.

Deterministic Vs

Randomised Algorithm.

Deterministic Algorithm	Randomized Algorithm.
1- Input is an ordinary number	Input is random number.
2. Infinite loop is unexpected	Infinite loop is expected.
3. Gives same output for the same input if run different times.	Gives different o/p if run more than once.

Probabilistic algorithm can be classified into mainly 4 types

- Numerical Algorithm.
- Monte Carlo Algorithm
- Las Vegas Algorithm, and
- Sherwood Algorithm.

Numerical Algorithm.

For some problems, finding the exact solution requires a long time, randomness can be used to find an

16 approximate numerical solution of such problems.

These algorithms produces approximate results and the quality of the solution can be improved if we run the algorithm for a longer time.

Monte Carlo Algorithm

There are some problems for which we need to find out the exact answer.

An approximate answer is meaningless for such types of problems.

For example:- In decision problems, the answer is either 'yes' or 'no'.

There is no scope for an approximate answer for these problems.

The probabilistic algorithms that are used to solve these kinds of problems are called Monte-Carlo algorithms.

Monte carlo algoms always gives an answer but the answer is not necessarily correct.

17 By running the algorithm for a longer time, the probability of correctness can be increased.

Las Vegas Algorithm

Las Vegas algorithms may or may not find an answer but whenever it finds an answer, the answer is correct.

Sherwood Algorithms.

These algorithms are used when we have a deterministic algorithm that shows different performances for the worst case and the average case.

By introducing an element of randomness, the behaviour of the algorithm may be made insensitive to the input data.

The sherwood algorithms reduce the gap in the time performance of the worst-case behaviour and the average case behaviour of the algorithm.

The sherwood algorithms gives an answer and the answer is correct.

24

Monte Carlo Algorithms.

These algorithms fail with some probability, but we can't tell when it fails.

If the algorithm produces a yes/no answer and the failure probability is significantly less than $\frac{1}{2}$, we can reduce the probability of failure by running it many times and taking a majority of the answers.

Karger's min cut algorithm is an example of a monte carlo algorithm.

Amplification of Stochastic Advantage.

Stochastic advantage is the advantage obtained from running a probabilistic algorithm.

Let p be the probability that an algorithm is correct.

If $p \leq \frac{1}{2}$, then there is no advantage in running the algorithm.

The advantage of the algorithm is $p - \frac{1}{2}$.

As we know a Monte Carlo algorithm occasionally makes mistakes but find the correct answer with high probability.

Let p be a real number such that $0 < p < 1$, we say Monte Carlo algorithm is p -correct if it returns a correct answer with probability at least p .

It is possible to reduce the error probability by allowing the algorithm to run more times.

This is called as amplification of stochastic advantage.

There are two types of algorithms.

① Brandom algorithms

Miller Rabin ~~com~~ primality testing algorithm is an example for brandom algorithm.

If Miller Rabin (n) says false, then n is definitely not prime.

If Miller-Rabin (n) says, true then n might be prime.

Here we have 100% certainty about one of the answer.

(2) Unbiased algorithm

These are algorithms where both answers are "may be".

There is no certainty about any answer.

Advantage of Biased algorithm

- Given a $3/4$ correct biased algorithm and we run the program 3 times.

- Probability of being correct.

$$P' = 1 - (1 - 3/4)^3$$

$$= 1 - (1/4)^3$$

$$= 1 - 1/64$$

$$= \frac{63}{64} = .984$$

$$\text{Advantage} = P - 1/2$$

$$= .984 - .5$$

$$= .484$$

$$P = 1 - (1 - P)^K$$

Las Vegas Algorithms.

These are algorithms that run for an unpredictable amount of time but always succeeds.

These algorithms fails with some probability, but we can tell when it fails.

In particular, we can run it again until it succeeds, which means that we can eventually succeed with probability 1.

Las Vegas algorithms are algorithms that never return an incorrect result, but may not produce results at all on some runs.

Robin Hood Eggset.

The performance of Las Vegas algorithms is not better than that of the corresponding deterministic algorithm in average case.

Instances that took a long time deterministically are solved much faster.

But instances on which deterministic algorithm was particularly good

50 are slowed down to average by Las Vegas algorithm.

That is, the Las Vegas algorithm steal time from the ~~rich~~ rich instances - those that were solved quickly by the deterministic algorithm, to give it to the poor instances.

We call this the "Robin Hood Effect".

Randomized Sorting:

Randomization in sorting is applied in the belief that the difference between the worst case and the average case behaviour can be reduced by making use of randomness.

In randomized quick sort, we select the pivot element for partitioning ~~as~~ randomly.

The expected runtime is $O(n \log n)$.

In randomized quick sort algorithm instead of using $A[r]$ as the pivot, we will use a randomly

51

chosen element from the subarray $A[p \dots r]$.

We do so by exchanging element $A[r]$ with an element chosen at random from $A[p \dots r]$.

This modification is which we randomly sample the range $p \dots r$, ensure that the pivot element $x = A[r]$ is equally likely to be any of the $r - p + 1$ elements in the subarray.

Because the pivot element is randomly chosen, we expect the split of the input array to be reasonably well balanced on average.

The changes to Partition and Quicksort are small.

In the new partition procedure, we simply implement the swap before actually partitioning.

Randomized-Partition (A, p, r)

$i \leftarrow \text{Random}(p, r)$

*exchange $A[r] \leftrightarrow A[i]$

return Partition (A, p, r)

The new quicksort calls Randomized-Partition in place of Partition.

52

Randomized - QuickSort (A, p, r)

if $p < r$

then $q \leftarrow$ Randomized - Partition (A, p, r)

Randomized - QuickSort ($A, p, q-1$)

Randomized - QuickSort ($A, q+1, r$).